Distributed Sort-Merge Join Using Key Range Shuffling

Minh Phan, Soumya Sistla, Trung Le
Group 3

1 Introduction

Merge join is often superior to hash join when inputs share the same join column and interesting orderings can be leveraged [4]. In Spark [9], merge joins are implemented by default using hash-based partitioning of the join keys for the shuffle step, which can lead to workload imbalance when the join key distribution is skewed. We examine an alternative way of shuffling based on key range. By using statistics on key distribution, we propose a key-range partitioning strategy that aims to achieve better workload balance across partitions.

2 Background

In distributed query engines such as Apache Spark, two common strategies for joining large datasets are hash joins and sort-merge joins. Hash joins build a hash table on one input table (typically the smaller one) and probe it with rows from the other. Sort-merge joins sort both input tables then merge them in order. In many cases, join inputs are already sorted—either because of indexing, a previous sort operation, or how the data was written. As interesting ordering [4] happens, a sort-merge join can skip the expensive sorting step and simply merge the datasets, making it more efficient than a hash join.

In this case, the sort-merge join consists of two main steps:

- Shuffle: Spark redistributes the data so that rows with the same join key end up in the same partition across both datasets.
- Merge: Within each partition, Spark performs a linear merge of the two sorted datasets to produce the joined output.

We focus on optimizing the shuffle step. In many real-world scenarios, join keys are skewed—for example, joining on attributes like person age or product sale date. **Skewness** presents a major challenge during shuffling, as it can lead to data being unevenly distributed across partitions, causing performance bottlenecks and task imbalance.

Spark partitions data during a shuffle using a simple hash-based partitioner. When join keys are heavily skewed this is cause workload imbalance. Figure 2 shows how Spark's default partitioning assigns workload when join keys are heavily skewed toward the lowest and highest values.

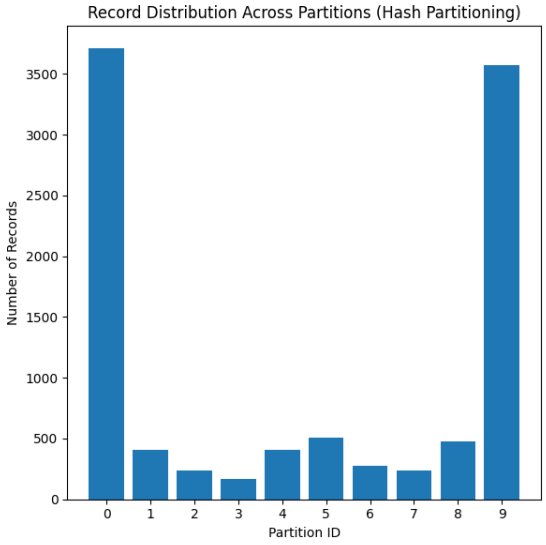

Figure 1: Workload skew from Spark's hash partitioner

In Spark 3.0, Spark Adaptive Query Execution (AQE) dealt with this issue by dynamic work re-balancing. After shuffling, if the engine realizes that a partition is too big, it can split these large partitions into smaller ones and redistribute them across available executors. This improves load balancing at the cost of additional re-shuffling.

We argue that key range partitioning is a suitable approach for handling this type of workload. Prior work has also explored the use of key distribution statistics to generate more efficient query plans [1], however, these approaches use one-time calculation and lacks the knowledge of new data.

3 Design

Figure 2 illustrates the overall workflow of a sort-merge join using key-range partitioning. The core idea lies in the *compute key range* step, where partition boundaries are determined based on key distribution.

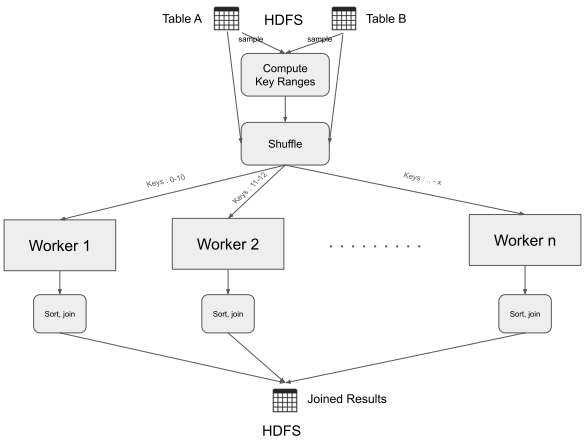

Figure 2: Merge Join Workflow with key-range shuffling

Key-range partitioning involves two steps: collecting join key statistics and generating a partitioning plan accordingly.

3.1 Obtaining Join Key Statistics

There are two different approaches to obtain key statistics.

- **Maintain join key distribution statistics within the storage system:** Statistics can be initially collected via a full scan of each table. As data is updated or queried, they can be incrementally maintained, similar to incremental view maintenance techniques in databases. This approach is very efficient and especially necessary with keys that occur frequently in joins.

- **Estimate statistics from a small data sample:** If the join key is rarely used or developers are unwilling to modify the system, statistics can be estimated from a subset of the data to avoid the cost of scanning the full table. This approach requires minimal changes to the system but may be ineffective if the sampled statistics do not accurately reflect the true distribution.

The output of this step is a mapping from each key value to its corresponding record count.

3.2 Partitioning Strategies for Skew-Aware Join

To evenly distribute join workloads across P partitions, we experimented with two partitioning strategies that leverage global key frequency statistics to address data skew: *greedy key-range partitioning* and *load-based greedy assignment*.

3.2.1 Key-Range Partitioning

Our first approach adopts a greedy key-range partitioning algorithm. The idea is to divide the sorted list of $(\text{key}, \text{frequency})$ pairs into contiguous key ranges, each assigned to a partition such that the total number of records per partition is approximately equal. This is done by computing the total number of records and dividing it by P to obtain the target load per partition. As we iterate through the sorted key list, we accumulate keys until the load exceeds the threshold, at which point we finalize the current key range and begin the next.

This method ensures that each partition is defined by a continuous range of keys, which can be used to guide data redistribution prior to the join. While simple and effective for moderately skewed data, we found that this approach struggles when key distributions are highly skewed—i.e., when a few keys dominate the total record count.

Algorithm 1 Greedy Key-Range Partitioning

Require: Sorted list of key frequencies: $(k_1, c_1), \ldots, (k_n, c_n)$
Require: Number of partitions: P
Ensure: List of key ranges $(\text{pid}, k_{\text{start}}, k_{\text{end}})$ Hash (random partitioning) Range (value-based splitting) Data Throughput Metric Hash Partitio

$T \leftarrow \sum_{i=1}^{n} c_i$ ▷ Total number of records
$L \leftarrow T \div P$ ▷ Ideal load per partition
Initialize empty list R for partition ranges
$s \leftarrow 1, w \leftarrow 0, p \leftarrow 0$ ▷ Start index, weight, partition ID
for $i = 1$ to n **do**
    $w \leftarrow w + c_i$
    if $w \geq L$ and $p < P - 1$ **then**
        Append (p, k_s, k_i) to R
        $p \leftarrow p + 1, s \leftarrow i + 1, w \leftarrow 0$
    end if
end for
if $s \leq n$ **then**
    Append (p, k_s, k_n) to R ▷ Remaining keys
end if
return R

3.2.2 Greedy Load-Balancing Assignment

To better handle extreme skew, we implemented a greedy load-balancing partitioning strategy that directly assigns each key to the partition with the lowest current load. Rather than creating contiguous key ranges, this method builds a key-to-partition mapping by iterating over the key-frequency pairs and assigning each key to the partition with the fewest cumulative records assigned so far. This results in a bin-packing–style distribution that more evenly spreads hot keys across workers.

The resulting key-to-partition map is then broadcast to all workers, who use it during the data redistribution phase. This strategy is explicitly skew-aware and does not rely on key ordering, making it more robust in the presence of Zipf-like distributions. The tradeoff is that the broadcasted mapping can become large for datasets with many distinct keys, increasing memory and communication overhead.

Algorithm 2 Greedy Load-Balancing Partitioning

Require: Sorted list of key frequencies: $(k_1, c_1), \ldots, (k_n, c_n)$
Require: Number of partitions: P
Ensure: Key-to-partition map: $M[k] \rightarrow$ pid
1: Initialize partition loads: $L[0 \ldots P{-}1] \leftarrow 0$
2: Initialize empty key-to-partition map: $M \leftarrow \{\}$
3: **for** $i = 1$ to n **do**
4: $(k, c) \leftarrow (k_i, c_i)$
5: pid $\leftarrow \arg\min_j L[j]$ ▷ Choose least loaded partition
6: $M[k] \leftarrow$ pid
7: $L[\text{pid}] \leftarrow L[\text{pid}] + c$
8: **end for**
9: **return** M

4 Evaluation

4.1 Experiment in Spark

To simulate realistic skewed join workloads, we generated synthetic datasets for two tables—left and right—with configurable row counts, key domain sizes, and skew using the Zipf distribution. The degree of skew is controlled independently for each table via a Zipf parameter, allowing exploration of both symmetric and asymmetric key distributions. Keys are clipped to a fixed domain to ensure a bounded join key space. Each table contains a unique row identifier, a join key, and a random payload column. To emulate distributed settings, the generated tables are split evenly into a configurable number of CSV partitions, representing data shards across multiple nodes.

We initially benchmarked our implementation against three baseline strategies:

- Default Hash Join (Spark's default strategy)
- AQE Merge Join, where Spark Adaptive Query Execution was enabled and forced to use a merge join
- Key-Range Join using repartitionByRange(), Spark's built-in range partitioner

Table 1: Benchmark

Shuffle Strategy	Run Time (s)	Shuffle Write	Shuffle Read
Default Hash Join	43	682.2 KiB	682.2 KiB
AQE Merge Join	50	652.4 KiB	652.4 KiB
Key Range Partitioning(Spark)	32.2	682.2 KiB	682.2 KiB
Our Greedy Partitioning	**46**	**1.6 MiB**	**1.8 MiB**
Greedy Partitioning w/o Stats comp.	**40**	**1.6 MiB**	**1.8 MiB**

From the results, we observed that computing statistics at runtime introduces a redundant overhead, especially as input sizes grow. However, since key distribution changes slowly or incrementally over time in most large datasets, we recognized an opportunity to compute the global statistics once, store them (e.g., in HDFS metadata or a catalog table), and incrementally update them whenever tables are appended to or updated.

For example, an append to the input dataset can be followed by a lightweight job that updates the global histogram by merging the new key frequencies with the existing ones. This avoids recomputation and improves overall pipeline efficiency.

With this optimization, we decoupled statistics computation from the main job and re-benchmarked the runtime:

As shown in Table 1, eliminating stats recomputation reduced runtime, making the approach practical for repeated analytical queries over large, slowly evolving datasets.

4.2 Experiment outside of Spark

Since modifying Spark's internal shuffling mechanism is non-trivial, we designed a complementary experiment using the classic MapReduce paradigm to compare hash-based versus key range-based shuffling strategies under controlled conditions.

Dataset: We generated a synthetic dataset using a custom Python script with controlled skew and partitioning characteristics. The dataset consists of two relational tables: Left and Right, each containing 10,000 rows, with 100 distinct join keys. The distribution of join keys in both tables follows a Zipfian distribution with parameters s=1.2 for both the Left and Right datasets. This induces moderate skew. Each table is partitioned into 12 separate

files, mimicking distributed data storage across partitions as commonly found in Hadoop and Spark environments.

Setup: We deployed HDFS across three nodes in CloudLab but mistakenly ran the MapReduce job locally on a single node due to a misconfiguration that we discovered too late. Our MapReduce implementation is written in Java and uses a custom partitioner — either a hash-based or key-range-based strategy. For the key-range partitioner, we assume access to optimal key ranges determined using the algorithm described in Section 3.

We then evaluate and compare the performance of hash partitioning versus key-range partitioning in the context of a merge join task enforcing 3 reducers in Table 2.

Metric	Hash Partitioning	Key-Range Partitioning
Job Duration (s)	36.39	38.30
Reducers	3	3
Output Records	17,308,019	17,308,019
Reducer 0 Output (%)	4%	24%
Reducer 1 Output (%)	2%	1%
Reducer 2 Output (%)	94%	75%
HDFS Bytes Written	609 MB	841 MB

Table 2: Comparison of Hash vs. Key-Range Partitioning in MapReduce Merge Join

Table 2 shows that key-range partitioning achieves a more balanced distribution of workload across reducers, though this comes at the expense of increased data shuffling. While sort-merge join with hash partitioning appears faster overall in Table 2, this is somewhat misleading because the reducers were executed sequentially. As shown in Table 3, if reducers were executed in parallel, the reduce phase would actually be faster with key-range partitioning.

Reducer ID	Hash Partitioning (s)	Key-Range Partitioning (s)
0	1.83	6.196
1	1.21	0.846
2	29.61	26.540

Table 3: Reducer runtimes

So over these experiment we can see that key-range partitioning might help in some cases but require careful calculation and implementation as it could cause shuffling cost to increase.

Given that our dataset is heavily skewed toward a single key, we recognize that our key-range partitioning approach may still be naive. While the algorithm ensures an even distribution of key ranges, it does not account for the frequency of individual keys — particularly when one key dominates. Since all identical keys must reside in the same partition to preserve correctness during the join, this can lead to significant load imbalance, even with ”optimal” ranges.

5 Related Work

Our approach builds on insights from prior work in handling data skew in relational databases and maintaining materialized views incrementally. Below, we summarize the most relevant related work.

Partitioning Techniques: Classical relational databases such as Oracle and PostgreSQL support range and hash partitioning to optimize query execution [6, 8]. Earlier work like Volcano [3] also emphasized partition-aware parallelism at the query engine level. These strategies often rely on key statistics to handle skew and improve plan quality.

Incremental View Maintenance: The idea of incrementally updating query results as underlying data changes has been studied extensively in database systems [7] [5]. Techniques such as delta queries and materialized view maintenance aim to reduce recomputation by only processing the affected data. In our context, we leverage similar ideas to maintain lightweight, up-to-date join key statistics that guide partitioning decisions.

Adaptive Query Execution: Apache Spark's Adaptive Query Execution (AQE) framework [2] dynamically adjusts join strategies and shuffle partition sizes at runtime using collected statistics. While AQE improves performance for hash-based joins, it does not natively support key-range partitioning or sort-merge-specific optimizations. Our work complements AQE by targeting scenarios where preserving key order and balancing range-based workload is crucial.

6 Future work

Possible future directions are:

- Integrate key-range based partitioning with Spark AQE framework.
- Maintaining lightweight and up to date key statistics during ingestion or via background task during operations.

These directions will make our approach applicable in real workloads.

Work Distribution: We developed this idea together, and alternate on setting up clusters, doing experiments and writing up reports. Soumya implemented the base stages for stats distribution and pipeline. Trung developed the key range partitioning algorithm. Minh implemented the Map Reduce code.

References

[1] Y. Chronis, T. Do, G. Graefe, and K. Peters. External merge sort for top-k queries: Eager input filtering

guided by histograms. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pages 2423–2437. ACM, 2020.

[2] Databricks. Adaptive query execution: Speeding up spark sql at runtime, 2020. Accessed: 2025-03-05.

[3] G. Graefe. Encapsulation of parallelism in the volcano query processing system. In *Proceedings of the 1990 ACM SIGMOD International Conference on Management of Data*, pages 102–111. ACM, 1990.

[4] G. Graefe. Query evaluation techniques for large databases. *ACM Comput. Surv.*, 25(2):73–169, June 1993.

[5] G. Graefe, F. Halim, S. Idreos, H. Kuno, and S. Manegold. Concurrency control for adaptive indexing. In *Proceedings of the VLDB Endowment (PVLDB)*, volume 5, pages 656–667. VLDB Endowment, 2012.

[6] Oracle Corporation. Oracle database concepts: Partitioning. `https://docs.oracle.com/en/database/oracle/oracle-database/`, 2024. Accessed: 2025-05-07.

[7] PostgreSQL Global Development Group. Incremental view maintenance. `https://wiki.postgresql.org/wiki/Incremental_View_Maintenance`, 2023. Accessed: 2025-05-07.

[8] PostgreSQL Global Development Group. Postgresql documentation: Table partitioning. `https://www.postgresql.org/docs/current/ddl-partitioning.html`, 2024. Accessed: 2025-05-07.

[9] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker, and I. Stoica. Spark: Cluster computing with working sets. In *Proceedings of the 2nd USENIX conference on Hot topics in cloud computing*, pages 10–10. USENIX Association, 2010.